

REVTRACK

Sistem realtime, IoT dan agentic AI

Fondasi yang matang untuk 1.000+ kendaraan dan pengembangan pipeline pasar

SYSTEM DESIGN / TECHNOLOGY REVIEW

Kajian end-to-end: pilihan stack, kapasitas, latency, data, hosting, keamanan, agent, biaya dan rencana implementasi.

STATUS	OWNER / PEMBACA	DIPERBARUI
Usulan untuk kajian	Founder dan tim RevTrack	3 Oktober 2026

Dasar kajian

Dokumentasi primer vendor dan proyek; pemeriksaan terbatas repo RevTrack; perhitungan kapasitas dengan asumsi eksplisit. Bukan hasil load test atau janji SLA.

Cara membaca

Mulai dari keputusan dan target, lalu ikuti aliran data sampai operasi. Referensi [Sxx] dapat diklik. Bookmark PDF membantu berpindah bab.

PETA BACA

Isi laporan

01 Keputusan arsitektur	3
02 Tujuan, batas, dan kondisi repo	4
03 Realtime harus punya definisi	5
04 Kapasitas 1.000 kendaraan	6
05 Batas sistem dan aliran data	7
06 Perangkat, GPS, dan MQTT	8
07 Kontrak data dan PostgreSQL	9
08 Konsistensi, replay, dan retensi	10
09 Broker: pilih satu sesuai tugas	11
10 Komponen mutakhir yang relevan	12
11 Go, Rust, Python, C++ dan OCaml	13
12 Flutter yang responsif dan jujur	14
13 Tim agent untuk RevTrack	15
14 Dots, Hermes, dan cara integrasi	16
15 Pipeline pasar dan eksekusi trading	17
16 Blockchain dan pengelolaan kunci	18
17 Keamanan dari perangkat sampai agent	19
18 Availability dan pemulihan	20
19 Sizing awal untuk eksperimen produksi	21
20 VPS, database, dan layanan hosting	22
21 Biaya: pisahkan fakta dan estimasi	23
22 Mac tools dan maintenance jangka panjang	24
23 Validasi sebelum menyebut production-ready	25
24 Urutan implementasi dan keputusan terbuka	26

Referensi primer dan catatan metodologi berada di bagian akhir. Harga, lisensi, versi serta ketersediaan perlu diverifikasi lagi saat keputusan pembelian.

Keputusan arsitektur

Bangun fondasi yang matang, lalu tambah komponen hanya ketika batasnya terlihat dalam pengukuran.

Rekomendasi inti: Linux yang masih didukung, Go, PostgreSQL + PostGIS, MQTT/TLS untuk perangkat, HTTPS/WebSocket untuk Flutter, serta worker terpisah untuk laporan dan AI. pgvector ditambahkan ketika pencarian berbasis dokumen benar-benar digunakan. Targetnya mudah dipelihara oleh tim kecil dan dapat berkembang melewati 1.000 kendaraan.

Tahap	Pilihan	Batas yang diterima
Testing HP	1 server Go + PostgreSQL terkelola + HTTPS	Belum high availability; bukan kapasitas produksi yang dijamin.
Pilot perangkat	Broker MQTT + ingestion + DB + live gateway	Uji perangkat, interval kirim, recovery dan biaya data.
1.000+ unit produksi	Komponen kritis redundan; antrean durable bila perlu; retensi bertingkat	Sizing berdasarkan event/detik, fan-out dan hasil benchmark.
Trading nyata	Deployment, identitas, ledger dan executor terpisah	Agent tidak melewati risk engine atau memegang kunci tanpa batas.

Dua pilihan hosting yang jelas

DigitalOcean + managed PostgreSQL Standard adalah jalur pilot yang relatif sederhana. AWS dengan RDS PostgreSQL dan IoT Core menjadi kandidat produksi saat kebutuhan perangkat, isolasi jaringan dan failover menguat. Gunakan satu region yang dekat sumber data dan pengguna; ukur dulu Singapura versus lokasi lain. Jangan memecah jalur telemetry ke beberapa cloud tanpa alasan.

Status dokumen: usulan teknis untuk kajian. Angka kapasitas adalah perhitungan asumsi; angka latency adalah target uji. Tidak ada benchmark produksi, pembelian hosting, migrasi database, atau deployment yang diklaim selesai.

Rujukan: S01 · S02 · S03 · S35 · S38 · S39

Tujuan, batas, dan kondisi repo

Yang harus dicapai

- Aplikasi di HP mengakses API HTTPS saat Mac pengembang mati; layanan server berjalan mandiri.
- Posisi dan status unit diperbarui tanpa menunggu proses AI; kualitas dan umur data terlihat oleh operator.
- Data tenant terisolasi; perangkat dapat dicabut aksesnya satu per satu; ada audit dan prosedur pemulihan.
- Fondasi protokol, schema, observability dan deployment dapat dipakai ulang oleh produk trading.

Bukti dari repo RevTrack saat kajian

Backend Go saat ini merupakan demo dengan fixture, token pengguna/perangkat statis, dan snapshot JSON opsional. Riwayat dibatasi 500 titik per unit. Flutter memuat ulang armada dan telemetry tiap sekitar lima detik. Implementasi ini berguna untuk menguji UI dan kontrak awal, tetapi belum menjadi pipeline IoT atau database produksi.

Berkas yang diperiksa: `services/api/internal/demo/store.go`, `persistence.go` dan `http.go`; `apps/mobile/lib/main.dart`, `widgets/telemetry_panel.dart`; `scripts/dev.py`; `Dockerfile` serta script build Android. Container dan build APK bukan pengganti migrasi PostgreSQL, autentikasi nyata, backup, atau pengujian perangkat.

Asumsi dan hal yang belum ditentukan

Perhitungan memakai 1.000 perangkat aktif bersamaan, operasi 24 jam, payload rata-rata 500 byte, serta interval 1/5/10 detik. Ini lebih konservatif daripada hanya 1.000 kendaraan unik yang aktif sebentar per hari. Model tracker, protokol, kualitas SIM, jumlah operator, masa retensi, exchange dan data feed belum ditentukan.

Di luar cakupan: high-frequency trading mikrodetik, jaminan profit, GPS tanpa error, kendali mesin kendaraan sungguhan, dan blockchain settlement dengan waktu final absolut. Rancangan tidak boleh menjanjikan hal-hal itu.

Teknologi lama yang tidak mendapat patch bukan pilihan aman. Tetapkan versi mayor yang didukung, uji pembaruan, lalu pasang patch keamanan secara rutin. PostgreSQL memberi dukungan lima tahun per versi mayor.

Rujukan: S01

03 / SLO DAN ANGGARAN LATENCY

Realtime harus punya definisi

Pisahkan delay pengiriman, umur data, akurasi sensor, dan availability. Angka “99% akurat” tidak menggambarkan keempatnya. Gunakan p50/p95/p99 pada histogram yang sama dan nyatakan kondisi pengukuran serta interval waktunya.

Ukuran	Target awal yang diusulkan	Definisi
Delay sample → layar	$p95 \leq 2$ detik; $p99 \leq 5$ detik pada koneksi sehat	Dari timestamp GNSS valid sampai aplikasi foreground merender update.
Umur posisi bergerak	99% pengamatan ≤ 10 detik pada operasi terhubung	Termasuk waktu menunggu sample berikutnya; berbeda dari delay satu pesan.
Ingestion internal	$p99 \leq 300$ ms pada steady state	Validasi sampai durable commit; diukur, bukan dijamin dari pilihan bahasa.
Ketersediaan API	99,9% per bulan sebagai sasaran awal	$\pm 43,2$ menit error budget / 30 hari; tidak mencakup kualitas radio perangkat.
Akurasi koordinat	Error dalam meter + percentile + jenis lingkungan	Uji referensi lapangan; accuracyM dari sensor tetap estimasi, bukan bukti.

Contoh alokasi delay, bukan angka vendor

Untuk mode kirim 1–2 detik, anggarkan radio/uplink 0,2–1,5 detik, broker/ingestion 0,05–0,3 detik, commit/fan-out 0,05–0,5 detik, dan render 0,05–0,2 detik. Sisakan headroom untuk variasi. Menjumlahkan p99 setiap tahap tidak otomatis memberi p99 end-to-end; ukur keseluruhan jalur dengan ID event yang sama.

Interval kirim 10 detik sudah menghabiskan hampir seluruh target freshness 10 detik. Mode 5 detik lebih ekonomis untuk fleet overview; mode 1–2 detik dapat diaktifkan ketika bergerak/diikuti operator. Tetapkan ambang stale per mode, bukan satu angka universal.

Simpan recorded_at, received_at, persisted_at, sent_at serta sequence. Gunakan waktu UTC dari perangkat yang tervalidasi dan jam server tersinkron. Ukur offset jam; jangan menyimpulkan delay negatif berarti jaringan sangat cepat. Periode offline tetap dilaporkan sebagai coverage gap.

Rujukan: S14 · S33

Kapasitas 1.000 kendaraan

Rumus: $\text{event/detik} = \text{perangkat aktif} \div \text{interval kirim}$. $\text{Event/hari} = \text{event/detik} \times 86.400$. Ukuran berikut memakai GB desimal dan payload 500 byte sebelum framing, TLS, indeks, WAL, replikasi, atau backup.

Interval	Event/detik	Event/hari	Payload/hari	Payload/30 hari
1 detik	1.000	86,40 juta	43,20 GB	1.296 GB
5 detik	200	17,28 juta	8,64 GB	259,2 GB
10 detik	100	8,64 juta	4,32 GB	129,6 GB

Penyimpanan fisik PostgreSQL harus diukur dengan schema dan indeks nyata. Untuk reservasi awal saja, pakai skenario 2–4× payload untuk heap + indeks, lalu tambahkan WAL, replica, backup dan ruang vacuum secara terpisah. Faktor ini bukan benchmark. Pada interval 5 detik, 30 hari raw JSON kecil pun sudah jauh di atas database 8 GB.

Burst dan fan-out lebih berbahaya daripada angka rata-rata

Jika 1.000 perangkat pada interval 5 detik offline 10 menit, backlog mencapai 120.000 event. Mengurasnya dalam 60 detik menambah 2.000 event/detik di atas 200 event/detik normal. Desain admission control dan reconnect jitter, lalu beri jalur live prioritas agar replay tidak membuat posisi terbaru ikut terlambat.

Sebanyak 100 operator yang masing-masing berlangganan 1.000 unit setiap detik menghasilkan 100.000 update/detik keluar. Filter subscription berdasarkan tenant, grup dan viewport; batasi jumlah unit per klien. Mode overview dapat menggabungkan update tiap 1–2 detik; detail satu unit lebih cepat. Ukur byte/detik dan penggunaan CPU di HP.

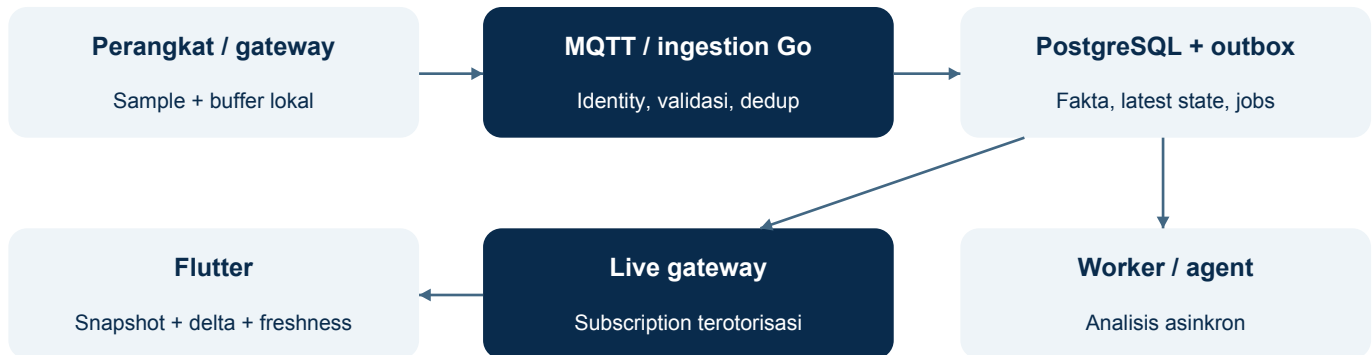
Retensi yang diusulkan untuk pilot

- Hot: posisi terbaru semua unit dan raw telemetry 7 hari; sesuaikan kebutuhan investigasi.
- Warm: agregat perjalanan/energi per menit dan event penting 90 hari, sesuai kebutuhan produk.
- Cold: arsip terkompresi dalam object storage dengan lifecycle dan aturan penghapusan.
- Audit keamanan/transaksi memiliki kebijakan sendiri. Jangan menghapus data relevan hanya demi target biaya.

Sizing 1.000 kendaraan tidak sama dengan 1.000 pesan/detik. Ukur juga jumlah sensor per sample, query historis, memakai dashboard, durasi aktif dan tingkat reconnect.

05 / PROPOSED ARCHITECTURE

Batas sistem dan aliran data



Stream durable dapat ditambahkan sebelum projector saat kebutuhan meningkat.

Tiga jalur dengan prioritas berbeda

Jalur live menangani validasi, status terkini dan update Flutter. Jalur durable menyimpan fakta, menjamin replay dan mendukung laporan. Jalur analisis mengonsumsi fakta yang sudah tersedia dan berjalan dengan kuota tersendiri. Kegagalan model AI tidak boleh menghentikan penerimaan telemetry.

Pada pilot, ingestion dan API dapat berupa modul di satu binary Go; worker tetap bisa dijalankan sebagai proses lain. Pisahkan service ketika profil CPU, kebutuhan scaling, atau batas keamanan berbeda. Jangan memecah setiap tabel menjadi microservice.

Pilihan commit yang harus konsisten

Baseline: commit telemetry + perubahan state + outbox dalam transaksi PostgreSQL, baru publikasikan update. Saat menambah stream durable, acknowledgement di ingress berarti event sudah tersimpan di stream; status “tersimpan di database” adalah tahap berikutnya. UI, audit dan metrik harus mengetahui perbedaannya.

Pisahkan domain RevTrack dan trading pada akun/project atau setidaknya deployment, secret, database dan batas akses berbeda. Reuse kode infrastructure dan event envelope; jangan berbagi identitas admin atau menaruh semua data sensitif di satu cluster demi kemudahan.

Rujukan: S08 · S15 · S17

Perangkat, GPS, dan MQTT

Pilih perangkat sebelum mengunci broker

Tidak semua tracker berbicara MQTT atau mendukung TLS. Banyak perangkat memakai protokol TCP/UDP vendor. Daftar Traccar menunjukkan banyaknya variasi ini; gateway decoder dapat dipertimbangkan setelah model unit diketahui. IMEI adalah identitas, bukan rahasia autentikasi. Untuk perangkat lama tanpa TLS, gunakan jalur jaringan terbatas bila tersedia dan gateway yang diisolasi; risikonya tetap berbeda dari mTLS end-to-end. [S13]

Kriteria perangkat: dokumentasi protokol, timestamp GNSS, sequence, buffer offline, pengaturan interval, status fix, accuracy/HDOP jika tersedia, upgrade firmware terverifikasi, serta pencabutan credential. Antena dan pemasangan diuji di kendaraan. GNSS multi-band/multi-constellation dan IMU bisa dievaluasi; RTK hanya dipilih bila kebutuhan meter/submeter, koreksi, biaya dan kondisi lapangan mendukung.

Urutan penerimaan event

- Perangkat membuat event ID stabil, nomor urut per boot/session, dan mencatat sample ke buffer persisten.
- Broker memverifikasi credential unik perangkat dan ACL topic; tenant ditentukan dari registrasi server.
- Ingestion memvalidasi schema, timestamp, koordinat, satuan, batas ukuran dan rate per perangkat.
- Event diterima secara durable; deduplikasi tidak bergantung pada koneksi TCP yang sedang hidup.
- Projector memperbarui latest state hanya jika event lebih baru; histori dapat menerima backlog yang sah.
- Perangkat menghapus buffer sesuai kontrak acknowledgement yang benar-benar disepakati.

MQTT QoS 1 dapat menghasilkan duplikat. PUBACK broker tidak otomatis membuktikan database atau Flutter sudah menerima pesan. QoS 2 pun tidak menjadikan efek bisnis lintas database/exchange exactly-once. Gunakan session expiry, Last Will, reconnect exponential backoff dan jitter sesuai kemampuan perangkat. [S08, S09]

GPS.gov memberi contoh smartphone sekitar radius 4,9 meter di langit terbuka, bukan jaminan tracker di jalan kota. Tampilkan status fix, umur data dan estimasi akurasi. Map matching dapat membantu tampilan, tetapi raw coordinate harus tetap tersedia. [S14]

Rujukan: S08 · S09 · S10 · S11 · S13 · S14

Kontrak data dan PostgreSQL

Gunakan PostgreSQL sebagai sumber kebenaran data operasional. PostGIS menangani lokasi/geofence; pgvector menyimpan embedding dokumen ketika diperlukan. Go mengakses database melalui pgx; sqlc dapat menghasilkan kode bertipe dari SQL agar query tetap eksplisit dan bisa diperiksa. [S02, S03, S25, S26]

Entitas	Isi utama	Aturan penting
devices / credentials	Device, tenant, firmware, credential status	Binding identitas ditentukan server; credential dapat dicabut.
telemetry_raw	Event ID, waktu GNSS, seq, posisi, speed, sensor	Partition waktu; null berarti unknown; provenance wajib.
vehicle_latest	Satu row per unit; latest timestamps	Conditional update agar event lama tidak memundurkan state.
trips / alerts	Hasil aturan deterministik + referensi event	Versi aturan dan evidence disimpan; bisa dihitung ulang.
outbox / jobs	Event, attempt, lease, next retry	Commit bersama perubahan bisnis; worker idempotent.
audit / memberships	Aktor, tenant, izin, aksi dan hasil	Scope diverifikasi; tidak menyalin token ke log.

Envelope contoh, bukan schema produksi final

```
schema_version, event_id, device_session_id, sequence
recorded_at, received_at, tenant_id_from_identity
vehicle_id_from_registry, latitude, longitude
speed_kph, accuracy_m, energy_percent_nullable
source, firmware_version, trace_id
```

API awal: GET /v1/fleet untuk snapshot; GET /v1/vehicles/:id/telemetry dengan cursor dan batas waktu; WSS /v1/live untuk update; POST /v1/reports untuk job asinkron. Ini kontrak usulan, bukan endpoint yang semuanya sudah ada. Payload live memuat event ID/versi dan timestamp agar klien dapat resync.

Tenant enforcement wajib di query dan authorization. Bila memakai RLS, runtime bukan pemilik tabel, superuser atau role BYPASSRLS; gunakan konteks tenant yang aman per transaksi dan uji isolation pada connection pool. Pisahkan role migrasi dari runtime. Index utama telemetry: tenant + vehicle + recorded_at, ditambah indeks spatial sesuai query nyata.

Rujukan: S02 · S03 · S04 · S25 · S26 · S45

Konsistensi, replay, dan retensi

At-least-once + efek bisnis idempotent

Gunakan kunci unik yang stabil: device/session/sequence atau event ID dari sumber tepercaya. Jangan hanya mengingat duplikat di RAM. Unique constraint pada tabel berpartisi mempunyai batas terkait partition key; desain dedup registry atau scope identitas sesuai horizon replay. Uji duplikat lintas hari, reboot perangkat, dan pengiriman ulang sesudah timeout.

Satu transaksi menulis telemetry yang diterima, latest state yang memenuhi urutan, serta outbox. Relay dapat mengirim outbox dua kali bila crash setelah publish; consumer harus tahan terhadap itu. “Exactly once” pada broker tidak mencakup seluruh efek eksternal. Koreksi data memakai event/versi baru, bukan mengubah fakta lama tanpa audit.

Late data bukan otomatis data berbahaya

Event lama yang sah dapat menambah histori tanpa menggeser posisi live. Event di luar batas jam/retensi masuk quarantine dengan alasan, tidak hilang diam-diam. Bedakan timestamp perangkat, received_at server dan sequence; nomor urut yang reset membutuhkan session/boot identity. Replay memisahkan traffic live dan kuota backfill.

Pool, partition, dan backup

Gunakan pool koneksi dengan batas global, statement timeout, lock timeout dan query cancellation. PgBouncer membantu ketika jumlah worker/koneksi bertambah; transaction pooling mempunyai batas fitur session dan harus diuji dengan driver. Partition harian/mingguan dipilih dari ukuran serta pola query; siapkan partition masa depan sebelum pergantian waktu. [S04, S06]

Database managed mengurangi kerja rutin, bukan menggantikan restore drill. Self-host PostgreSQL memerlukan backup yang teruji, misalnya pgBackRest dengan WAL archive. RPO adalah berapa banyak data yang boleh hilang; RTO adalah waktu pemulihan. Keduanya harus disepakati dan dibuktikan dengan eksperimen. [S07]

Jangan menjalankan laporan berat pada primary saat ingest sibuk. Mulai dengan agregat; tambah replica atau warehouse jika metrik menunjukkan kebutuhan. Replica bisa tertinggal sehingga pembacaan segera setelah write perlu diarahkan dengan benar.

Rujukan: S04 · S05 · S06 · S07 · S15 · S16 · S17

Broker: pilih satu sesuai tugas

Teknologi	Kegunaan umum / RevTrack	Kapan dipilih
Mosquitto	Broker MQTT ringan untuk koneksi perangkat	Pilot atau edge; satu instance bukan cluster HA.
AWS IoT Core	MQTT managed + identity perangkat	Saat operasi fleet dan certificate lifecycle lebih penting daripada kontrol broker sendiri.
EMQX	Broker MQTT dengan fitur pengelolaan/cluster	Evaluasi berbayar untuk fleet; versi dan lisensi cluster harus diperiksa.
RabbitMQ	Queue pekerjaan, routing, ack/retry	Laporan/notifikasi/command workflow yang memerlukan antrean kerja.
NATS JetStream	Stream durable, replay dan consumer	Kandidat utama bus internal ringan ketika fan-out mulai banyak.
Apache Kafka	Log terpartisi dan banyak consumer independen	Jika replay panjang, CDC dan ekosistem streaming menjadi kebutuhan utama.

Rekomendasi awal: MQTT → Go → PostgreSQL/outbox. Tambahkan NATS JetStream untuk memisahkan ingestion, live fan-out dan analitik ketika diperlukan. Jika tim sudah berpengalaman dengan Kafka atau kebutuhan replay/connector besar, pilih Kafka sebagai pengganti bus tersebut. Jangan menjalankan Kafka, NATS dan RabbitMQ sekaligus untuk tugas yang sama.

Desain consumer lebih penting daripada nama broker

Tentukan partition/subject key per tenant dan unit, batas unacked message, retry dengan backoff, quarantine/dead-letter dan retention. Consumer mengakui pesan setelah efek durable yang diinginkan selesai. Pastikan key berfrekuensi tinggi tidak membuat satu partition panas. Urutan per kendaraan diperlukan; urutan global seluruh fleet biasanya tidak.

JetStream mendukung persistence, replay dan replikasi; konfigurasi tiga node harus ditempatkan pada failure domain berbeda untuk manfaat nyata. Kafka memberikan semantik transaksional di ruang lingkup tertentu, bukan jaminan order exchange atau commit database eksternal. EMQX mendokumentasikan lisensi komersial untuk clustering versi terkait. [S12, S15, S16]

Redpanda layak menjadi pembanding Kafka jika tim menilai kecocokan API dan biaya operasi. Jangan menyamakan “Kafka-compatible” dengan perilaku identik; periksa lisensi edisi, dukungan fitur, upgrade dan recovery sebelum memilih. [S64]

Rujukan: S08 · S11 · S12 · S15 · S16 · S17 · S64

10 / TAMBAHAN BERDASARKAN BUKTI

Komponen mutakhir yang relevan

Penilaian kematangan berikut adalah keputusan arsitektur untuk proyek ini, bukan sertifikasi keamanan. Komponen tambahan dipasang setelah ada kebutuhan dan tim yang dapat mengoperasikannya.

Komponen	Manfaat konkret	Syarat masuk
TimescaleDB	Hypertable/time-series dan agregasi telemetry	Benchmark dibanding partition PostgreSQL biasa; periksa versi extension serta lisensi fiturnya.
ClickHouse	Analitik kolumnar untuk histori GPS atau tick pasar	Query analitik mengganggu OLTP; ingest batch dan retensi sudah dirancang.
Valkey	Cache, rate limit, latest-state fan-out	Bisa dibangun ulang; bukan ledger saldo atau satu-satunya penyimpanan event.
Qdrant	Pencarian vektor sebagai service khusus	pgvector sudah diukur dan tidak memenuhi target recall/latency/isolasi.
Temporal	Workflow durable, retry, timer dan approval	Pekerjaan multi-step panjang sering gagal dan perlu dipulihkan secara eksplisit.
Apache Flink	Event-time windows dan stateful stream processing	Agregasi lintas stream, late events dan skala melampaui worker sederhana.
Open Policy Agent	Aturan authorization terpusat	Banyak service/agent perlu kebijakan yang seragam, diaudit dan dites.
Firecracker	Isolasi microVM untuk eksekusi kode agent	Benar-benar menjalankan kode tidak tepercaya; tersedia operasi Linux/KVM yang sesuai.

Valkey mereplikasi secara asinkron dalam penggunaan umumnya; failover dapat kehilangan write yang belum tereplikasi. Cache harus mempunyai jalur rebuild. Temporal tidak menjadikan efek API eksternal exactly-once: activity yang retry tetap perlu idempotency dan reconciliation. Flink juga menambah state, checkpoint dan pekerjaan operasional. [S20, S21, S22]

Belum perlu sekarang: service mesh, database multi-region active-active, GPU untuk setiap agent, atau kernel-bypass networking. Untuk target 1–10 detik, bottleneck awal biasanya radio, interval kirim, backlog, query, fan-out dan panggilan model, bukan kekurangan teknologi yang paling baru.

Rujukan: S18 · S19 · S20 · S21 · S22 · S23 · S30 · S56

Go, Rust, Python, C++ dan OCaml

Pilihan	Peran yang tepat	Batas keputusan
Go	API, ingestion, websocket gateway, worker	Default RevTrack; observasi CPU, GC, alokasi dan lock sebelum rewrite.
Rust	Decoder/proses berintensitas CPU atau komponen dengan kebutuhan memory safety	Pisahkan service; tambah ketika manfaat terukur melebihi kompleksitas build.
Python	AI, riset, fitur statistik, backtest offline	Worker terpisah dengan batas waktu/RAM; jangan menahan ingestion sinkron.
C++	SDK vendor/komponen native dan engine khusus	Isolasi proses, fuzzing decoder, sanitizers dan review memory safety.
OCaml	Komponen domain yang memang cocok dan tim kuasai	Kontrak HTTP/gRPC jika dukungan library memadai; tidak wajib untuk crypto.

Kontrak menghubungkan bahasa

REST/JSON dengan OpenAPI cocok untuk Flutter dan API eksternal. MQTT untuk perangkat yang mendukungnya. gRPC/Protobuf berguna antarservice dengan schema bertipe; pilih setelah profiling dan dukungan SDK diperiksa. Tidak ada jaminan gRPC selalu 10× lebih cepat daripada REST. Payload, jaringan, connection reuse dan implementasi lebih menentukan. [S24]

Kebijakan schema dan timeout

Schema berversi; penambahan field harus kompatibel. Nyatakan satuan dan timezone, gunakan integer scaled/decimal untuk harga dan kuantitas, bukan float biner untuk ledger uang. Setiap pemanggilan service mempunyai deadline, payload cap dan cancellation; retry hanya untuk operasi yang aman atau punya idempotency key.

Harga “lebih cepat” juga berupa biaya maintenance: bahasa baru membutuhkan ownership, pipeline build, scanning dependency, on-call dan pengujian lintas versi. Bangun image multi-architecture di CI bila dev memakai Apple Silicon tetapi server x86. Pin artifact release; jangan menggunakan tag latest sebagai strategi upgrade.

Untuk target detik, Go dapat tetap menjadi fondasi. Rust/C++ adalah alat optimasi atau integrasi khusus; menambah bahasa tidak otomatis mempercepat jaringan seluler atau respons model AI.

Rujukan: S24 · S25 · S26

Flutter yang responsif dan jujur

Snapshot dahulu, kemudian delta

Aplikasi mengambil snapshot armada dari HTTPS API, lalu berlangganan delta WebSocket yang scoped ke tenant/grup/unit. Sertakan cursor atau versi state. Bila koneksi putus atau ada gap, klien mengambil snapshot baru; jangan menganggap semua pesan terdahulu selalu bisa di-replay dari memory gateway.

Pada server, simpan subscription dengan batas jumlah unit, ukuran buffer dan kecepatan kirim. Klien lambat dapat menerima latest-state yang sudah digabung, tetapi event audit/alert kritis memakai jalur durable dan fetch tersendiri. Menggabungkan update tampilan tidak boleh menghapus histori sensor.

Anggaran UI dan koneksi

- Update hanya widget dan marker yang berubah; bedakan stream armada, detail unit dan grafik historis.
- Batasi titik polyline dan sample chart; gunakan downsampling untuk tampilan dengan sumber raw tetap tersimpan.
- Interpolasi posisi untuk animasi diberi batas waktu. Saat data stale, hentikan gerak prediktif dan tampilkan waktu terakhir.
- Reconnect memakai backoff + jitter dan refresh token; saat aplikasi kembali foreground lakukan resync.
- Push notification diperlukan untuk alert saat aplikasi background. WebSocket mobile tidak menjamin tetap aktif ketika OS menidurkan aplikasi.

API reporting dan pekerjaan agent

POST job mengembalikan ID dan status, bukan menunggu seluruh laporan selesai. Flutter menerima progress faktual melalui stream atau polling job. Gunakan request cancellation, timeout dan hasil parsial yang jelas. Angka di tabel/chart berasal dari query snapshot yang tercatat; model hanya menjelaskan hasil yang dapat dirujuk.

Perubahan pertama yang relevan di repo: mengganti polling lima detik pada halaman live, menambah metadata freshness, serta memisahkan mode fixture dari koneksi perangkat nyata. Animasi atau indikator "live" saja tidak membuktikan realtime.

Rujukan: S24 · S49

Tim agent untuk RevTrack

Gunakan pool worker yang menjalankan pekerjaan saat dibutuhkan. Identitas “agent untuk setiap kendaraan” boleh berupa konfigurasi dan konteks; tidak berarti 1.000 proses agent selalu hidup atau setiap sample GPS memanggil LLM. Aturan deterministik memicu analisis ketika ada peristiwa bermakna.

Peran	Input dan hasil	Izin default
Fleet analyst	Agregat fleet → ringkasan anomali dengan bukti	Read-only pada data tenant.
Maintenance assistant	Histori servis → usulan inspeksi	Membuat draft; persetujuan sebelum perubahan operasional.
Report writer	Snapshot SQL → narasi + tabel rujukan	Tidak menghitung angka bisnis bebas dari prompt.
Support/driver assistant	Konteks terbatas → jawaban/draft pesan	Tidak otomatis menelepon/mengirim pesan.
Supervisor	Routing tugas dan penggabungan hasil	Bukan administrator DB, cloud atau perangkat.

Orchestration yang terkendali

Untuk integrasi produk, saya memilih backend agent Python dengan OpenAI Agents SDK sebagai kandidat pertama karena aplikasi dapat memiliki tool, state dan approval sendiri. SDK dan runtime agent adalah lapisan yang lebih cepat berubah daripada PostgreSQL; lakukan version pinning, eval dan canary sebelum upgrade. Multi-agent dipakai hanya ketika hasilnya lebih baik daripada satu agent dengan tools. [S47, S48]

Go mengirim job dengan tenant scope yang diverifikasi, references ke evidence dan budget. Worker mempunyai batas concurrency, jumlah langkah, biaya token, wall-clock timeout dan jumlah retry. Cache jawaban berdasarkan snapshot dan izin; batalkan job jika izin dicabut. Circuit breaker menghentikan panggilan model yang gagal berulang tanpa mematikan monitoring fleet.

Panggilan LLM tidak dijamin selesai dalam 1–10 detik. UI dapat memberi acknowledgement cepat, menampilkan progress, dan mengirim hasil ketika selesai. Ukur time-to-first-output dan time-to-final-answer terpisah. Jangan memperlakukan token streaming pertama sebagai tugas yang sudah selesai. [S49]

Contoh budget asumsi: 1 analisis/unit/jam × 1.000 unit = 24.000 job/hari. Jika tiap job rata-rata 2.000 token input dan 500 output, hasilnya 48 juta input dan 12 juta output token/hari, sebelum retry. Supervisor dan spesialis dapat menggandakan biaya.

Rujukan: S31 · S47 · S48 · S49

Dots, Hermes, dan cara integrasi

Dots: rujukan pengalaman, akses perlu diverifikasi

Dokumentasi resmi OpenAI menggambarkan dots sebagai agent cloud yang selalu tersedia dan dapat bekerja di berbagai tools; rollout bergantung akun yang memenuhi syarat. Halaman yang diperiksa tidak membuktikan adanya widget Flutter atau hak white-label untuk menanamkan produk dots secara langsung. Karena itu desain RevTrack tidak menggantungkan rilis pada kemampuan embed tersebut. [S46]

Hermes Agent: kandidat eksperimen yang terisolasi

Hermes dari Nous Research mendokumentasikan tools, memory, skills, messaging gateway, subagent dan MCP. Untuk RevTrack, tempatkan di backend sebagai worker opsional di balik API berizin; Flutter hanya mengakses API aplikasi. Tidak perlu memasukkan terminal, filesystem atau secret Hermes ke APK. Verifikasi versi, interface integrasi, lisensi, isolasi session dan proses upgrade sebelum produksi. [S50]

Dokumentasi Hermes menjelaskan perbedaan approval menurut terminal backend, termasuk pemeriksaan command yang dilewati pada backend container tertentu. Maka jangan menganggap dialog approval runtime sebagai satu-satunya pembatas. Izin tool dan pemeriksaan aksi tetap ditegakkan oleh Go/policy gateway di luar proses agent. [S51]

Batas eksekusi yang saya sarankan

- Container/VM terpisah dari database; non-root, resource quota, egress allowlist dan secret yang minimal.
- Tidak memasang Docker socket host, credential cloud admin atau direktori host sensitif ke sandbox.
- Tool khusus seperti get_vehicle_status dan draft_report; tidak memberikan raw SQL, arbitrary URL fetch atau shell tanpa kebutuhan.
- Memory dan retrieval dipisahkan per tenant; setiap query vector diberi filter izin sebelum hasil masuk model.
- Skill buatan agent menjadi draft; perubahan tool/skill production ditinjau, dites dan ditandatangani melalui release pipeline.

Container bukan jaminan kebal escape atau salah konfigurasi. Untuk menjalankan kode tidak tepercaya dari banyak tenant, evaluasi isolasi lebih kuat seperti microVM. Firecracker membutuhkan infrastruktur host yang sesuai; bukan dependency yang tinggal ditambahkan ke Flutter. [S56]

Rujukan: S31 · S46 · S47 · S50 · S51 · S56

Pipeline pasar dan eksekusi trading



Market data masuk melalui connector resmi exchange/provider. Simpan source timestamp, received_at, sequence/trade ID dan simbol yang dinormalisasi. Order book perlu snapshot + delta serta gap detection sesuai kontrak feed. Coinbase dan Binance mendokumentasikan perilaku WebSocket yang harus diikuti per produk; jangan memakai satu algoritme reconnect untuk semua feed tanpa pengujian. [S52, S53]

Target 1–10 detik punya batas penggunaan

Target ini dapat cocok untuk dashboard, alert atau sebagian strategi berfrekuensi rendah. Itu belum cukup untuk menyatakan cocok bagi arbitrase cepat atau HFT. Market-data latency, waktu keputusan, broker acknowledgement dan fill adalah metrik berbeda; order diterima tidak sama dengan order terisi. Saham juga membutuhkan feed dengan cakupan dan entitlement yang benar; data gratis tidak otomatis mewakili seluruh pasar. [S54]

Executor menyimpan intent/order state secara durable, memakai client order ID bila didukung, dan merekonsiliasi status ketika timeout. Jangan mengulang order secara buta setelah koneksi putus. Gunakan decimal/scaled integer untuk nilai finansial, aturan tick/lot size, serta ledger dengan audit dan rekonsiliasi terhadap broker.

Agent menghasilkan analisis atau proposal. Risk engine deterministik memeriksa freshness, batas notional/posisi, jumlah order, slippage policy dan penghentian trading. Paper trading dan shadow mode mendahului uang nyata. Desain ini tidak menjanjikan akurasi prediksi atau keuntungan.

Rujukan: S31 · S52 · S53 · S54

Blockchain dan pengelolaan kunci

Blockchain adalah sumber event dan tujuan transaksi tambahan, bukan pengganti PostgreSQL untuk seluruh bisnis. Simpan signature/hash, chain/network, slot/block, commitment/finality, waktu observasi dan status reconciliation. Connector menangani retry serta perubahan status tanpa mengubah fakta pengamatan sebelumnya.

Solana: cepat terlihat belum tentu final

RPC Solana membedakan commitment processed, confirmed dan finalized. Pilih sesuai aksi bisnis: dashboard boleh menampilkan status awal dengan label, sementara penyelesaian bernilai tinggi memerlukan kebijakan finality yang lebih ketat. Jangan menjanjikan semua transaksi finalized dalam 1–10 detik. Infrastruktur RPC, congestion dan status jaringan harus masuk monitoring. [S55]

Pisahkan kemampuan analisis, otorisasi dan signing

- Agent tidak menerima seed phrase, private key, atau kunci withdrawal exchange.
- Signer terpisah hanya menerima permintaan bertipe yang lolos policy, dengan batas tujuan, aset, nilai dan masa berlaku.
- Pilihan KMS/HSM atau custody diverifikasi untuk kurva, metode signing, latency, recovery dan dukungan chain yang sebenarnya.
- Pisahkan key testnet, staging dan produksi; inventaris, rotasi, pencabutan dan emergency response harus dilatih.
- Untuk trading exchange, nonaktifkan withdrawal pada API key jika tersedia; IP allowlist dan scope minimal bila provider mendukung.

Posisi OCaml, Rust dan C++

Rust dapat relevan untuk ekosistem Solana dan komponen native; OCaml dapat dipakai untuk domain/analisis yang tim kuasai. Pilihan blockchain tidak memaksa seluruh backend beralih bahasa. Connector berbasis kontrak membuat upgrade SDK/chain tidak mengubah modul armada atau login.

Seluruh tindakan bernilai harus punya policy di luar LLM, identitas aktor, approval sesuai risiko, batas waktu, dan catatan hasil. Security boundary harus tetap berlaku walau agent salah membaca dokumen, terkena prompt injection, atau mengulang tool call.

Rujukan: S31 · S55

Keamanan dari perangkat sampai agent

Risiko	Kontrol utama	Bukti yang harus diuji
Perangkat palsu / credential bocor	Credential per perangkat, ACL topic, revocation, rate cap	Device A tidak bisa publish sebagai B; revoked device ditolak.
Kebocoran tenant	Authorization server + scoped query/RLS + test	User/agent tenant A tidak melihat detail, vector atau report tenant B.
Prompt injection / tool abuse	Tool allowlist, schema validation, policy eksternal	Dokumen berisi perintah berbahaya tidak memperluas izin.
Supply-chain / dependency	Pin digest/version, scan, provenance, review upgrade	Artifact sama dari build ke deploy; secret tidak ikut image.
Ransomware / penghapusan	Backup terisolasi + retention + restore drill	Akun aplikasi tidak bisa menghapus backup; recovery terbukti.
DoS / biaya agent	Quota per tenant, bounded queues, timeouts, budget cap	Satu tenant tidak menghabiskan koneksi, token atau worker.

HTTPS/mTLS hanya melindungi bagian transport. Tetap periksa payload, identitas, authorization dan replay. Cloud admin memakai MFA kuat dan akun individual; akses database/admin melalui jalur privat atau allowlist yang ketat. Tidak ada database credential di Flutter. Public Mapbox token mempunyai fungsi berbeda dari secret server.

Untuk login pengguna, gunakan provider OIDC yang terpelihara. Keycloak cocok jika tim memang ingin mengoperasikan identity sendiri; alternatif managed mengurangi kerja on-call. Google login tetap harus diverifikasi server dan dipetakan ke membership/role tenant. Token demo saat ini tidak layak menjadi auth produksi. [S29]

Log menghilangkan token, key dan payload sensitif. Trace AI dapat memuat lokasi, prompt atau dokumen; minimalkan/redact sebelum dikirim ke monitoring. Tetapkan retensi lokasi, akses support dan proses penghapusan sesuai kebutuhan bisnis serta wilayah operasi. [S32]

Backup Object Lock dapat membantu membuat objek tidak mudah dihapus selama retensi yang disetel. Mode, bucket, hak akses dan akun backup tetap harus dirancang; fitur ini bukan pengganti backup konsisten dan latihan restore. [S34]

Rujukan: S10 · S29 · S30 · S31 · S32 · S34 · S40 · S45

Availability dan pemulihan

Satu server murah tidak memberikan high availability

Untuk produksi, gunakan minimal dua instance API/gateway pada failure domain berbeda jika platform mendukungnya, load balancer sehat, database primary + standby, dan broker/stream dengan konfigurasi redundansi yang sesuai. Satu data center/regional outage tetap memerlukan disaster recovery di lokasi lain. Tidak semua provider menawarkan pemisahan zone seperti AWS; periksa definisi failure domain, jangan menganggap dua VM otomatis independen.

RDS Multi-AZ instance menggunakan standby untuk failover, bukan replica baca tambahan. Replikasi sinkron mempunyai tradeoff latency/availability. PostgreSQL async replica dapat tertinggal. RPO=0 bukan janji universal ketika seluruh region atau semua salinan rusak. [S05, S35]

Mode kegagalan	Perilaku yang direncanakan	Target awal uji
Satu worker mati	Lease berakhir; job diambil ulang secara idempotent	Tidak hilang/duplikat efek bisnis.
DB/broker tidak sehat	Backpressure; buffer perangkat; jangan mengaku tersimpan	Tidak membuang data diam-diam.
Model AI gagal	Telemetry dan dashboard tetap berjalan	Agent degraded, ada retry budget.
Region gagal	Restore/deploy ke region DR yang disiapkan	RPO ≤ 5 menit, RTO ≤ 60 menit sebagai usulan, bukan hasil uji.
Perangkat reconnect massal	Jitter, quota replay, live diprioritaskan	Backlog terkuras tanpa melanggar freshness live.

Runbook minimum

Siapkan prosedur disk hampir penuh, credential bocor, broker restart, database failover, restore snapshot/WAL, rollback aplikasi dan penghentian trading. Setiap prosedur mempunyai owner, metrik pemicu, langkah eksekusi, bukti pemulihan dan waktu terakhir diuji. Backup di provider yang sama dengan akun admin yang sama masih mempunyai risiko bersama.

RPO/RTO di tabel adalah proposal untuk diskusi. Bila bisnis menuntut pemulihan lebih cepat, biaya replika, DR dan on-call harus dinaikkan. Restore dari backup saja berbeda dari failover otomatis.

Rujukan: S05 · S07 · S34 · S35 · S38

19 / CPU, RAM, DISK DAN AUTOSCALING

Sizing awal untuk eksperimen produksi

Asumsi sizing: 1.000 perangkat aktif, interval 5 detik, 200 event/detik steady state, raw hot 7 hari, payload 500 byte, dan subscription operator yang dibatasi. Spesifikasi berikut adalah titik mulai load test, bukan minimum universal atau konfigurasi yang sudah terbukti.

Komponen	Titik mulai eksperimen	Yang diukur / ditingkatkan
Go API + ingestion + live	2 instance, masing-masing 2–4 vCPU dan 4–8 GB RAM	CPU/GC, durable commit, koneksi aktif, outbound byte/detik dan latency p99.
PostgreSQL	Primary 4 vCPU / 16 GB + standby setara untuk HA	IOPS, WAL, locks, cache hit, query plan, replica lag dan failover.
Disk database	Mulai menguji kelas 300–500 GB SSD sesuai retensi/indeks	7 hari payload ≈60,48 GB; faktor fisik, WAL dan ruang bebas masih perlu diukur.
Broker MQTT	Managed IoT atau broker yang diuji untuk 1.000 sesi + reconnect	Session state, persistence, inflight, TLS handshakes dan lisensi HA.
JetStream, jika ditambahkan	3 node; uji 2 vCPU / 4 GB dan 50–100 GB disk per node	Retention, replication, quorum, write latency dan drain backlog.
Agent/report worker	Resource dan concurrency terpisah dari ingestion	Antrian job, token budget, waktu tugas dan akses tenant.

Contoh stream 72 jam pada interval 5 detik menyimpan sekitar 25,92 GB payload logis. Replikasi tiga salinan menjadi sekitar 77,76 GB agregat sebelum overhead; bukan 25,92 GB total cluster. Tambahkan metadata, indeks dan headroom. Pilih retensi sesuai kebutuhan replay, jangan menggandakan seluruh histori DB tanpa tujuan.

Scale API berdasarkan CPU, active connections dan p99. Scale consumer berdasarkan lag dan service time, dengan batas koneksi DB. Jangan menambah worker terus ketika database sudah penuh. Rate limit replay lebih aman daripada membiarkan reconnect storm memicu autoscaling tanpa batas.

Pada interval 1 detik, volume menjadi 5× skenario ini. Uji kapasitas baru dan biaya storage/messaging sebelum mengaktifkan mode tersebut untuk seluruh armada. Mode cepat dapat dibatasi pada kendaraan yang bergerak atau sedang dipantau.

Rujukan: S05 · S15 · S35

VPS, database, dan layanan hosting

Paket	Kapan cocok	Tradeoff yang harus diterima
DigitalOcean + managed PostgreSQL Standard	Pilot sampai produksi yang sudah dibenchmark	Operasi relatif sederhana; periksa region, extension dan definisi HA.
AWS EC2/ECS + RDS + IoT Core	Fleet dengan kebutuhan identity, jaringan dan operasi IoT serius	Lebih banyak konfigurasi; hitung messaging, NAT, log, storage dan egress.
Hetzner Cloud + PostgreSQL sendiri	Tim menguasai Linux/DB dan ingin kendali biaya	Patch, failover, WAL backup dan recovery menjadi tanggung jawab tim.
Supabase + server Go	Ingin PostgreSQL, auth/storage cepat tersedia	Atur RLS/grants; PITR dan compute memiliki biaya/ketentuan tersendiri.
Neon + server Go	Development dan workload PostgreSQL yang cocok dengan compute elastis	Telemetry nonstop tidak banyak mendapat manfaat scale-to-zero; verifikasi plan terkini.

Rekomendasi untuk keputusan pembelian

Mulai pilot dekat pengguna Indonesia, misalnya Singapura setelah pengukuran jaringan. Tempatkan API, broker dan DB sedekat mungkin. Jangan membeli server Eropa hanya karena lebih murah tanpa mengukur koneksi SIM Indonesia. Untuk trading pilih lokasi berdasarkan endpoint exchange/provider; kebutuhan itu bisa berbeda dari RevTrack.

DigitalOcean Standard mencantumkan PostGIS/vector pada daftar ekstensi, sedangkan Advanced memiliki daftar preinstalled yang lebih terbatas. Pilihan edisi harus mengikuti fungsi GPS. Supabase Pro mempunyai backup harian, tetapi PITR merupakan tambahan berbayar menurut halaman harga; jangan menganggap semua jenis recovery termasuk paket dasar. [S39, S44]

Panel deployment dan reverse proxy

Docker Compose cocok untuk awal; CI membangun image dan deploy artifact yang dipin. HAProxy cocok ketika membutuhkan load balancing HTTP/TCP. Caddy memudahkan HTTPS otomatis. Panel seperti Coolify boleh dipertimbangkan sebagai kenyamanan operator, tetapi admin panel juga perlu patch dan akses terbatas; bukan syarat sistem ini.

Tidak ada pembelian atau deployment dalam kajian ini. Sebelum memilih plan, verifikasi kuota dan harga checkout regional, dukungan extension versi target, opsi private networking, restore, egress dan dukungan teknis.

Rujukan: S27 · S28 · S35 · S39 · S40 · S42 · S43 · S44 · S62 · S63

21 / ANGGARAN DAN SENSITIVITAS

Biaya: pisahkan fakta dan estimasi

Harga publik diperiksa 3 Oktober 2026. USD, sebelum pajak. Angka server/database berikut adalah anchor harga, bukan jaminan kapasitas atau total tagihan.

Komponen dasar	Harga publik	Implikasi
DO Droplet 2 GB / 1 vCPU	US\$12/bulan	Titik awal server testing.
DO Droplet 4 GB / 2 vCPU	US\$24/bulan	Lebih banyak headroom; bukan benchmark ingest.
DO PostgreSQL satu node 1 GB	Mulai US\$15/bulan	Bukan HA; kapasitas storage/IO perlu dihitung.
DO PostgreSQL HA minimum	Primary US\$30 + standby US\$30/bulan	DB saja mulai US\$60, belum app/storage tambahan.
Supabase Pro dasar	Mulai US\$25/bulan	Compute, quota dan add-on memengaruhi total.

Server US\$12 + DB US\$15 = US\$27/bulan adalah konfigurasi testing. Untuk 1.000 kendaraan, terutama interval 1–5 detik, jangan memakai angka ini sebagai anggaran produksi. [S36, S37, S44]

Envelope perencanaan, bukan penawaran vendor

Sisihkan kisaran awal US\$200–800/bulan untuk eksperimen produksi 1.000 unit pada interval sekitar 5 detik, retensi hot terbatas, redundancy dasar dan observability terkendali. Ini asumsi budgeting yang harus diganti hasil sizing; dapat melampaui kisaran bila storage, messaging, HA, egress atau lisensi broker besar. Belum termasuk AI, SIM, perangkat, data pasar, Mapbox dan tenaga operasi.

Pada interval 5 detik: 518,4 juta publish per 30 hari. Jika tarif hipotetis adalah US\$1/juta unit billable, publish saja US\$518,40; ini ilustrasi sensitivitas, bukan tarif AWS Singapura. Broker managed dapat menagih koneksi, publish/delivery, ukuran bertingkat dan rules/action. Gunakan kalkulator region/plan dengan jumlah subscriber sebenarnya. [S41]

Persamaan biaya yang dipakai saat sizing

Total = compute + DB/IO/storage + broker/messages
 + egress + backup + monitoring + lisensi
 + AI tokens/tools + market data + SIM/perangkat

Biaya AI memakai jumlah job × token rata-rata × harga model, ditambah tools dan retry. Jangan menghitung 1.000 kendaraan sebagai hanya 1.000 panggilan AI per bulan. Batasi concurrency, budget tenant dan jumlah langkah agent.

Rujukan: S36 · S37 · S41 · S42 · S44

Mac tools dan maintenance jangka panjang

Tool	Dipakai untuk	Pilihan RevTrack
DataGrip	SQL IDE, navigasi schema dan pengembangan query	Pilihan utama bila perlu fitur lengkap; lisensi komersial untuk produk bisnis.
DBeaver Community	Client database lintas engine	Alternatif gratis yang layak untuk operasional awal.
TablePlus	Client native untuk kerja data sehari-hari	Pilih jika kenyamanan Mac lebih penting daripada IDE SQL penuh.
pgAdmin	Administrasi PostgreSQL, role, backup, query plan	Pendamping khusus PostgreSQL; akses server mode harus dibatasi.

Aplikasi GUI bukan database hosting. Gunakan identitas read-only sebagai default untuk produksi, koneksi TLS terverifikasi, dan akun terpisah untuk migrasi. Tombol “read-only” di UI bukan pengganti pembatasan role server. Audit pengaturan fitur AI/telemetry pada client sebelum memasukkan query atau data sensitif. [S57–S61]

Kebijakan release dan upgrade

- Buat daftar komponen, versi, lisensi, owner, tanggal akhir dukungan dan prosedur upgrade.
- CI: unit/integration test, schema compatibility, vulnerability scan, secret scan, image digest dan artifact yang dapat ditelusuri.
- Migrasi database memakai expand–migrate–contract agar app lama/baru dapat hidup selama rolling deploy.
- Patch rutin dijadwalkan; kerentanan aktif yang relevan ditangani lebih cepat dengan prosedur emergency change.
- Simpan infrastructure sebagai kode ketika bentuk deployment stabil; uji restore ke environment kosong.
- Tinjau quota, backup, biaya dan akses tiap bulan; latih failover/restore berkala dengan hasil tercatat.

Jangan membangun semua komponen sendiri hanya agar “bukan SaaS”. Database managed sering mengurangi risiko operasional bagi tim kecil. Self-host masuk akal jika ada keterampilan, waktu on-call, monitoring dan recovery yang benar-benar tersedia.

Kubernetes bukan syarat 1.000 kendaraan. Pindah ketika jumlah service, kebutuhan scheduling/rollout dan tim pengelola membenarkan kompleksitasnya; angka device saja bukan pemicu.

Rujukan: S01 · S57 · S58 · S59 · S60 · S61 · S62

23 / UJI BEBAN, KEAMANAN DAN FAILURE

Validasi sebelum menyebut production-ready

Gunakan generator Go untuk protokol perangkat/MQTT, k6 untuk API dan WebSocket, serta perangkat fisik untuk uji GNSS/radio. Uji dengan payload, tenant, indeks, retention dan koneksi TLS yang menyerupai produksi. Benchmark localhost tanpa persistence tidak cukup. [S33]

Uji	Skenario	Kriteria penerimaan usulan
Steady load	1.000 perangkat, interval 5 detik, 24 jam	p99 sample→layar ≤5 dtk pada koneksi sehat; error tercatat; backlog stabil.
Peak / headroom	1.000 event/detik + operator sesuai asumsi	Tidak ada silent loss; DB dan memory tidak terus naik tanpa batas.
Reconnect storm	120.000 replay event + 200/detik live	Live tetap mendapat prioritas; drain selesai dalam target yang diuji.
Crash/failover	Kill worker/API, restart broker, failover DB	Deduplikasi benar; recovery otomatis atau runbook memenuhi target.
Tenant security	Manipulasi tenant/unit ID dan retrieval AI	Tidak ada kebocoran; forbidden action ditolak di backend.
Agent abuse	Prompt injection, tool spoofing, runaway loop	Izin tidak meluas; budget/timeout bekerja; audit tidak memuat secret.
Trading safety	Feed gap, stale quote, timeout order, duplicate ack	Trading diblokir/reconcile; tidak mengirim order ganda secara buta.
Restore	Pulihkan ke environment kosong	Data tervalidasi; RPO/RTO aktual dicatat.

Metrik yang wajib terlihat

Event rate, reconnect rate, active device, data age p50/p95/p99, queue lag, redelivery, quarantine, commit latency, WAL growth, disk free, slow query, websocket lag, error rate dan AI job/token cost. Hindari vehicle_id sebagai label semua metrik Prometheus karena cardinality; detail unit disimpan di log/event store dengan kontrol akses.

“Tidak kehilangan data” dinilai dari event ID yang diakui durable versus data tersimpan/replayed, bukan hanya jumlah pesan yang pernah dikirim generator. Laporkan juga data yang memang belum pernah mencapai server saat perangkat offline.

Rujukan: S31 · S32 · S33

24 / ROADMAP YANG DAPAT DIEKSEKUSI

Urutan implementasi dan keputusan terbuka

Urutan	Hasil yang dibangun	Syarat selesai
1. Fondasi	Schema PostgreSQL/PostGIS, migrasi demo, identity pengguna, HTTPS	HP mengakses backend mandiri; tenant test dan backup/restore lulus.
2. Pilot IoT	Pilih tracker, decoder/MQTT, credential unik, telemetry envelope	10–50 unit uji lapangan; delay/accuracy/coverage dicatat.
3. Live + durable	WebSocket, latest-state, outbox, dedup, retention	UI resync benar; backlog tidak memundurkan posisi terkini.
4. Scale gate	Load test 1.000 unit, monitoring, failover, sizing	SLO dan biaya diukur; bottleneck tersisa diketahui.
5. AI terbatas	Agent laporan read-only dengan evidence dan budget	Eval, tenant isolation, injection test dan audit lulus.
6. Multi-agent / Hermes	Eksperimen terisolasi, tools terpilih, canary	Ada manfaat terukur; izin tetap di luar agent.
7. Trading domain	Connector data pasar, paper trading, risk engine, reconciliation	Shadow mode lulus; review terpisah sebelum uang nyata.

Keputusan yang dibutuhkan sebelum belanja produksi

- Berapa perangkat aktif bersamaan dan interval kirim saat bergerak/diam? Model dan protokol apa?
- Berapa operator simultan, unit per layar dan retensi raw yang benar-benar diperlukan?
- Berapa batas biaya bulanan termasuk AI, Mapbox, data SIM dan feed pasar?
- Siapa owner on-call, backup, patch dan incident response?
- Market/exchange mana, jenis strategi apa, hak penggunaan data apa, dan apakah aksi uang nyata diperlukan?
- Apakah izin/residensi data atau kontrak pelanggan membatasi region/provider?

Pilih fondasi sekarang: Go + PostgreSQL/PostGIS + HTTPS + MQTT sesuai perangkat. Buktikan pipeline 1.000 unit dahulu. Tambahkan bus, warehouse dan agent sesuai hasil pengukuran, bukan sebagai prasyarat setiap rilis.

Laporan ini tidak menetapkan “vendor kebal hacker”. Ketahanan datang dari konfigurasi yang dibatasi, patch yang dikelola, data yang dapat dipulihkan, dan tindakan agent yang selalu diperiksa sistem lain. Pola itu dapat dipakai ulang; risiko dan data RevTrack serta trading tetap dipisahkan.

Referensi 1

Sumber mendukung kemampuan produk dan fakta harga. Rancangan, target SLO, asumsi sizing dan roadmap merupakan analisis untuk RevTrack. Tidak ada klaim bahwa vendor menjamin hasil benchmark dalam laporan ini.

[S01] PostgreSQL: version support

<https://www.postgresql.org/support/versioning/>

[S02] PostGIS: spatial capabilities

<https://postgis.net/>

[S03] pgvector: vector search

<https://github.com/pgvector/pgvector>

[S04] PostgreSQL: table partitioning

<https://www.postgresql.org/docs/current/ddl-partitioning.html>

[S05] PostgreSQL: standby and replication

<https://www.postgresql.org/docs/current/warm-standby.html>

[S06] PgBouncer: pooling features

<https://www.pgbouncer.org/features.html>

[S07] pgBackRest: backup and restore

<https://pgbackrest.org/>

[S08] AWS IoT: MQTT semantics

<https://docs.aws.amazon.com/iot/latest/developerguide/mqtt.html>

[S09] AWS IoT: protocols and TLS

<https://docs.aws.amazon.com/iot/latest/developerguide/protocols.html>

[S10] AWS IoT: device certificates

<https://docs.aws.amazon.com/iot/latest/developerguide/x509-client-certs.html>

[S11] Eclipse Mosquitto

<https://mosquitto.org/>

[S12] EMQX: licensing FAQ

<https://www.emqx.com/en/content/license-faq>

Referensi 2

[S13] Traccar: tracker protocols

<https://www.traccar.org/protocols/>

[S14] GPS.gov: GPS accuracy

<https://www.gps.gov/gps-accuracy>

[S15] NATS JetStream

<https://docs.nats.io/reference/2.12/jetstream>

[S16] Apache Kafka: design

<https://kafka.apache.org/design/>

[S17] RabbitMQ: acknowledgements and confirms

<https://www.rabbitmq.com/docs/confirms>

[S18] ClickHouse: introduction

<https://clickhouse.com/docs/get-started/about/intro>

[S19] Tiger Data / Timescale

<https://www.tigerdata.com/>

[S20] Valkey: replication

<https://valkey.io/topics/replication/>

[S21] Temporal: workflow execution

<https://docs.temporal.io/workflow-execution>

[S22] Apache Flink: architecture

<https://flink.apache.org/what-is-flink/flink-architecture/>

[S23] Qdrant: overview

<https://qdrant.tech/documentation/overview/>

[S24] gRPC: concepts

<https://grpc.io/docs/what-is-grpc/core-concepts/>

Referensi 3

[S25] Go pgx driver

<https://github.com/jackc/pgx>

[S26] sqlc

<https://docs.sqlc.dev/en/latest/>

[S27] HAProxy

<https://www.haproxy.org/>

[S28] Caddy: automatic HTTPS

<https://caddyserver.com/docs/automatic-https>

[S29] Keycloak: OpenID Connect

<https://www.keycloak.org/securing-apps/oidc-layers>

[S30] Open Policy Agent

<https://www.openpolicyagent.org/docs>

[S31] OWASP: excessive agency

<https://genai.owasp.org/llmrisk/llm062025-excessive-agency/>

[S32] OpenTelemetry: security

<https://opentelemetry.io/docs/security/>

[S33] Grafana k6: thresholds

<https://grafana.com/docs/k6/latest/using-k6/thresholds/>

[S34] AWS S3: Object Lock

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>

[S35] RDS: Multi-AZ standby

<https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZSingleStandby.html>

[S36] DigitalOcean: Droplet pricing

<https://www.digitalocean.com/pricing/droplets>

Referensi 4

[S37] DigitalOcean: PostgreSQL pricing

<https://docs.digitalocean.com/products/databases/postgresql/details/pricing/>

[S38] DigitalOcean: PostgreSQL features

<https://docs.digitalocean.com/products/databases/postgresql/details/features/>

[S39] DigitalOcean: supported extensions

<https://docs.digitalocean.com/products/databases/postgresql/details/supported-extensions/>

[S40] DigitalOcean: secure database access

<https://docs.digitalocean.com/products/databases/postgresql/how-to/secure/>

[S41] AWS IoT Core pricing

<https://aws.amazon.com/iot-core/pricing/>

[S42] AWS RDS PostgreSQL pricing

<https://aws.amazon.com/rds/postgresql/pricing/>

[S43] Hetzner Cloud Singapore

<https://www.hetzner.com/cloud-singapore/>

[S44] Supabase pricing

<https://supabase.com/pricing>

[S45] Supabase: RLS

<https://supabase.com/docs/guides/database/postgres/row-level-security>

[S46] OpenAI: Meet dots

<https://learn.chatgpt.com/docs/dots>

[S47] OpenAI: Agents SDK

<https://developers.openai.com/api/docs/guides/agents/sdk>

[S48] OpenAI: agent orchestration

<https://developers.openai.com/api/docs/guides/agents/orchestration>

Referensi 5

[S49] OpenAI: latency optimization

<https://developers.openai.com/api/docs/guides/latency-optimization>

[S50] Hermes Agent documentation

<https://hermes-agent.nousresearch.com/docs/>

[S51] Hermes Agent: security

<https://hermes-agent.nousresearch.com/docs/user-guide/security/>

[S52] Coinbase: WebSocket best practices

<https://docs.cdp.coinbase.com/exchange/websocket-feed/best-practices>

[S53] Binance: Spot WebSocket streams

<https://developers.binance.com/docs/binance-spot-api-docs/web-socket-streams>

[S54] Alpaca: real-time stock feeds

<https://docs.alpaca.markets/us/docs/real-time-stock-pricing-data>

[S55] Solana RPC and commitment

<https://solana.com/docs/rpc>

[S56] Firecracker microVMs

<https://firecracker-microvm.github.io/>

[S57] DataGrip features

<https://lp.jetbrains.com/datagrip/features-overview/>

[S58] DataGrip licensing

<https://www.jetbrains.com/datagrip/buy/>

[S59] TablePlus

<https://tableplus.com/>

[S60] DBeaver Community

<https://dbeaver.io/about/>

Referensi 6

[S61] pgAdmin features

<https://www.pgadmin.org/features/>

[S62] Docker Compose service reference

<https://docs.docker.com/reference/compose-file/services/>

[S63] Neon: compute pricing announcement

<https://neon.com/blog/major-compute-price-reduction-on-neon>

[S64] Redpanda Cloud overview

<https://docs.redpanda.com/cloud-data-platform/get-started/cloud-overview/>